

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT: Artificial intelligence
PRACTICAL NO. 5

Aim:Support Vector Machines (SVM)

- Implement the SVM algorithm for binary classification.
- Train an SVM model using a given dataset and optimize its parameters.
- Evaluate the performance of the SVM model on test data and analyze the results.

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load the Iris dataset
df = pd.read_csv("Iris.csv")

# Select two classes for binary classification
df = df[df["Species"].isin(["Iris-setosa", "Iris-versicolor"])]

# Select two features for classification and visualization
X = df[["PetalLengthCm", "PetalWidthCm"]]
y = df["Species"]

# Split the dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
```

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT: Artificial intelligence

PRACTICAL NO. 5

)

Standardize the feature values

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

Define different SVM parameters for optimization

parameters = {

 "C": [0.1, 1, 10, 100],

 "gamma": ["scale", "auto"],

 "kernel": ["linear", "rbf"]

}

Find the best combination of parameters

grid_search = GridSearchCV(

 SVC(),

 parameters,

 cv=5,

 scoring="accuracy"

)

Train GridSearchCV

grid_search.fit(X_train, y_train)

Display the best parameters

print("Best Parameters:")

print(grid_search.best_params_)

Display the best cross-validation accuracy

print("\nBest Cross-Validation Accuracy:")

print(grid_search.best_score_)

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT: Artificial intelligence

PRACTICAL NO. 5

```
# Train the SVM model using the best parameters
```

```
model = grid_search.best_estimator_
```

```
# Predict the test data
```

```
y_pred = model.predict(X_test)
```

```
# Calculate test accuracy
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("\nTest Accuracy:")
```

```
print(accuracy)
```

```
print("\nTest Accuracy Percentage:")
```

```
print(accuracy * 100, "%")
```

```
# Confusion Matrix
```

```
print("\nConfusion Matrix:")
```

```
print(confusion_matrix(y_test, y_pred))
```

```
# Classification Report
```

```
print("\nClassification Report:")
```

```
print(classification_report(y_test, y_pred))
```

```
# Convert test labels into numeric values for plotting
```

```
classes = model.classes_
```

```
train_labels = np.where(y_train == classes[0], 0, 1)
```

```
test_labels = np.where(y_test == classes[0], 0, 1)
```

```
# Create a mesh grid for decision boundary
```

```
x_min = X_train[:, 0].min() - 1
```

```
x_max = X_train[:, 0].max() + 1
```

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT: Artificial intelligence

PRACTICAL NO. 5

```
y_min = X_train[:, 1].min() - 1
y_max = X_train[:, 1].max() + 1

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 500),
    np.linspace(y_min, y_max, 500)
)

# Predict each point of the mesh
Z = model.predict(
    np.c_[xx.ravel(), yy.ravel()]
)

Z = np.where(Z == classes[0], 0, 1)

Z = Z.reshape(xx.shape)

# Plot the SVM decision boundary
plt.figure(figsize=(9, 6))

plt.contourf(
    xx,
    yy,
    Z,
    alpha=0.3
)

# Plot training data
plt.scatter(
    X_train[:, 0],
    X_train[:, 1],
    c=train_labels,
    edgecolors="black",
    marker="o",
    label="Training Data"
)
```

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT: Artificial intelligence
PRACTICAL NO. 5

```
# Plot test data
# Warning fixed: removed edgecolors from marker "x"
plt.scatter(
    X_test[:, 0],
    X_test[:, 1],
    c=test_labels,
    marker="x",
    s=80,
    label="Test Data"
)

# Labels and title
plt.xlabel("Standardized Petal Length")
plt.ylabel("Standardized Petal Width")

plt.title("SVM Decision Boundary")

plt.legend()

plt.show()
```

OUTPUT:

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT: Artificial intelligence

PRACTICAL NO. 5

```
IDLE Shell 3.13.2
File Edit Shell Debug Options Window Help
Python 3.13.2 (tags/v3.13.2:4f8bb39, Feb  4 2025, 15:23:48) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: E:/AI/T071_AI_Prac.py =====
Best Parameters:
{'C': 0.1, 'gamma': 'scale', 'kernel': 'linear'}

Best Cross-Validation Accuracy:
1.0

Test Accuracy:
1.0

Test Accuracy Percentage:
100.0 %

Confusion Matrix:
[[10  0]
 [ 0 10]]

Classification Report:
              precision    recall  f1-score   support

 Iris-setosa      1.00      1.00      1.00        10
 Iris-versicolor  1.00      1.00      1.00        10

   accuracy      1.00      1.00      1.00        20
  macro avg      1.00      1.00      1.00        20
 weighted avg      1.00      1.00      1.00        20

>>>
```

